An RTEMS device driver for the Bosch BMP180 I2C pressure sensor

A. Furlan T. F. Costantini

September 7, 2026

Contents

NOTE: IT IS MANDATORY TO FILL IN ALL THE SECTIONS AND SUBSECTIONS IN THIS TEMPLATE

1 Project data

- Project supervisor(s): Emilio Corigliano
- The group delivering this project:

Last and first name	Person code	Email address
Furlan, Alberto	10730756	alberto2.furlan@mail.polimi.it
Costantini, Tommaso Fabrizio	10918262	tommasofabrizio.costantini@mail.polimi.it

- How the development tasks were subdivided among the members of the group:
  - **A. Furlan** — toolchain and board bring-up, the I2C bus and the first sensor readings, hardware validation, the host-side analysis and the testing campaign.
  - **T. F. Costantini** — the driver and its ioctl interface, the application and telemetry layers and the testing campaign.

  Both of us worked on the hardware and reviewed each other's changes.
- Public repository hosting the project source code:
  - Repository: `https://github.com/AlbertoFurlan20/ES2025`
  - Firmware: `C_src/` — driver, bus, application layer, telemetry.
  - Host analysis tooling and captured datasets: `E_analysis/`.
  - Version history and rationale for every change: `CHANGELOG.md`.

2 Project description

- **What is your project about?**

The project is a device driver, written from scratch, for the Bosch BMP180 digital barometric pressure and temperature sensor, running under **RTEMS 7** on an **STM32F4-Discovery** board (Cortex-M4, `arm/stm32f4` BSP). The sensor is an I2C slave at address `0x77`; the deliverable is the software stack that turns it into an ordinary RTEMS device node, `/dev/bmp180-0`, that any task can `open()` and `ioctl()`.

It took four modules rather than one, because the BSP does not provide everything the driver needs:

Module	Directory	What it is
`bmp180/`	`C_src/src/bmp180/`	the sensor driver — the scope of the assignment
`i2c/`	`C_src/src/i2c/`	a polled STM32F4 I2C master, since the BSP ships none
`bmp_app/`	`C_src/src/bmp_app/`	application layer: one sampler task owns the device
`telemetry/`	`C_src/src/telemetry/`	measurement stream out over USART2, operator commands in

The stack is complemented by a host-side Python package (`E_analysis/`) that captures the serial stream and computes the noise, timing and drift statistics used to validate the driver against the datasheet. No statistics are computed on the target: the board emits raw records and the numbers are derived on the host, so they can be recomputed without reflashing.

- **Why it is important for the AOS course?**

The project exercises, on real hardware, most of the topics the course covers about the boundary between an operating system and the devices it drives:

- *The driver model.* The driver plugs into the RTEMS generic I2C framework (`libdev/i2c`). A `bmp180_dev_t` has an `i2c_dev` as its first member so the framework can upcast safely, and registration publishes an IMFS node that owns the device, so the node's destroy handler frees it rather than the caller. Getting that ownership backwards is a use-after-free, which we hit (Section 3.4).
- *The system-call surface.* The public interface is a frozen ioctl ABI (`_IOR`/`_IOW` encodings, Section 2.1), which is the concrete form of the userspace/kernel contract discussed in the course.
- *Scheduling and priority.* The final design is three tasks at fixed priorities, with every consumer *below* the producer. Getting the interaction wrong cost a measured 2.70 s console blackout: a priority-2 task busy-looping on a failing ioctl starved every task below it.
- *Concurrency without blocking the real-time path.* The application layer publishes measurements through a seqlock rather than a mutex, so a reader can never delay acquisition.
- *Bounded blocking and fault containment.* Every I2C poll has a 5 ms deadline and every conversion wait a datasheet-derived maximum, so a disconnected or wedged sensor degrades into an error record instead of a system freeze.
- *Timing as a measurable property.* Because `rtems_task_wake_after(n)` blocks for between $n-1$ and n ticks, an unpadded conversion timeout can expire while the sensor is still within specification. That one tick of rounding was a stochastic defect in our first release, visible only in the noise statistics.

This is not an open source project contribution, so there is no upstream maintainer correspondence to report.

2.1 Design and implementation

The full source is in the repository under `C_src/`; what follows is the structure.

Layering. One folder per module, mirrored across `src/`, `inc/` and `tests/`. Only `inc/` is on the include path, never `inc/<module>/`, so every include names the module it reaches into (`#include "bmp_app/app.h"`). Dependencies only ever point downwards: `bmp180/` knows nothing of `bmp_app/`, and `bmp_app/` knows nothing of `telemetry/`. `init.cpp` and `constants.h` sit at the top because they belong to the image rather than to any module.

The bus driver (`i2c/`). The `arm/stm32f4` BSP ships no driver for the modern `<dev/i2c/i2c.h>` framework, so the sensor driver had nothing to sit on. We wrote a polled master bus driver that registers as `/dev/i2c-1`: peripheral clock gating, GPIO alternate function mux, CCR/TRISE clock math against `PCLK1` (16 MHz on this BSP) for 100 kHz standard mode, and start/address/data/stop sequencing. Every status poll is bounded by `I2C_POLL_TIMEOUT_US = 5000`. Everything that differs between I2C1, I2C2 and I2C3 is isolated in a `stm32f4_i2c_hw` descriptor; the transfer engine is instance-agnostic.

It also carries a *bus recovery* sequence. A CPU reset landing mid-transfer leaves the BMP180 holding SDA low waiting for clock pulses that never arrive; the peripheral latches BUSY and every transfer returns `-EBUSY`. Before this was added, only a power cycle cleared it. Recovery takes the pins away from the peripheral, drives up to nine SCL pulses by hand (eight data bits plus the ACK), issues a manual STOP, software-resets the peripheral and restores the configured clock. A line shorted to ground cannot be freed this way and is reported as a distinct error.

The sensor driver (`bmp180/`). `bmp180_register()` allocates the device through the framework, reads the chip ID register (`0xD0`, expecting `0x55`) and publishes the node. A registration that succeeds proves the wiring, not the calibration. The eleven 16-bit calibration coefficients are read once, lazily, on the first measurement and cached in the device context. `bmp180_unregister()` goes through `unlink()`, since the IMFS node owns the device.

The public interface exposed on the node is:

```
ioctl(fd, BMP180_IOCTL_READ_MEASUREMENT,  &result) // bmp180_measurement_t
ioctl(fd, BMP180_IOCTL_SET_OSS,           &oss)    // bmp180_oss_t
ioctl(fd, BMP180_IOCTL_GET_OSS,           &oss)    // bmp180_oss_t
ioctl(fd, BMP180_IOCTL_SOFT_RESET)                 // no payload
ioctl(fd, BMP180_IOCTL_SET_TEMP_INTERVAL, &ms)     // uint32_t
ioctl(fd, BMP180_IOCTL_GET_TEMP_INTERVAL, &ms)     // uint32_t
```

A measurement is returned as a `bmp180_measurement_t`: temperature in steps of $0.1\,^{\circ}$C and pressure in Pa. The Bosch compensation algorithm is implemented verbatim from datasheet section 3.5 (Figure 4) in `int32_t`/`int64_t` arithmetic, so the image carries no floating-point dependency in the measurement path.

Two design decisions inside the driver:

- *Conversion waits are polled, not slept through.* The driver sleeps the datasheet *typical* conversion time, then polls the SCO bit of `ctrl_meas` until the sensor reports the conversion finished, giving up at the datasheet *maximum* plus one tick of padding. Reaching the maximum is an error (`ETIMEDOUT`), not a slow sample.
- *Temperature is cached.* Pressure compensation needs a temperature, not a *fresh* temperature, and the datasheet notes temperature can be measured far less often than pressure. Re-reading it every cycle spent a whole extra 3–6 ms conversion on a quantity that moves in minutes. The interval is 1000 ms by default and settable per device; 0 restores one temperature conversion per measurement.

The application layer (`bmp_app/`). `bmp_app_sampler_task` is the only task that opens `/dev/bmp180-0`: it acquires back-to-back and publishes each measurement into a one-deep snapshot. Consumers call `bmp_app_read()`, which never blocks and adds no bus traffic, and change the mode or the temperature interval through setters the sampler applies at the top of its next cycle. `oss` therefore has exactly one writer, and no consumer holds a descriptor on the device.

The snapshot is a **seqlock**, not a mutex: the sequence counter is odd while a publish is in progress and even when the payload is stable, and readers retry. An RTEMS mutex held by a low-priority reader would block the sampler until the priority-inheritance handoff completed; here the reader simply retries and the writer never waits at all.

A successful read does not mean a fresh reading. A failed cycle does not publish, so a sensor that stops answering leaves the last good sample, with its last good timestamp, in place indefinitely. Consumers detect it by comparing `seq` against their previous read, or `t_us` against the current uptime, and learn the reason from `bmp_app_last_error()`.

The telemetry layer (`telemetry/`). `bmp180_telemetry_task` is a consumer like any other: it holds no descriptor and issues no ioctl. It subscribes to the sampler (`BMP_APP_EVENT_SAMPLE`), drives the validation profile through `bmp_app_set_oss()`, and writes records to USART2. The wire format is frozen at v1 and machine-readable:

Record	Meaning
`#BMP180 v1 fw=... temp_ms=...`	session header, written once before any record
`S <t_us> <t_cdeg> <p_pa> <oss>`	one sample
`E <t_us> <errno>`	an acquisition error, reported on the edge
`D <t_us> <lost>`	samples the consumer missed

The timestamp in an `S` record is the one the *sampler* took at acquisition, never the time the telemetry task happened to run, because the host derives the achieved rate and the jitter from differences between consecutive records. Errors are emitted on the edge: a disconnected sensor fails hundreds of times a second and would otherwise flood the console.

The control surface (`telemetry/control.cpp`). The same USART2 that carries telemetry out carries operator commands in, on `PA3`. `bmp180_control_task` owns the read side and is the only task that reads `stdin`. Like the telemetry task it is a consumer of `bmp_app`: a command becomes a `bmp_app_set_oss()` or a `bmp_app_request_soft_reset()`, applied by the sampler at the top of its next cycle. One command per line:

Line	Effect
`00`–`03`	switch to that oversampling mode
`R`	soft reset; the device returns to power-on defaults

Anything else is discarded. The task never writes to the console, since an echo or an error message would appear in every capture as a line the host parser has to skip; the console is put into canonical mode with echo off at startup for the same reason. Confirmation comes from the `oss` field of the next samples, which reports the mode they were taken at.

The first accepted command cancels the boot sweep (`telem_cancel_sweep`): the sweep steers `oss` itself and keys each block on getting the mode it asked for, a condition that can never match again once an operator sets the mode. A capture used for validation is one where no command was typed.

Tasks and priorities. The image runs three tasks after initialisation:

Task	Priority	Role
`SAMP`	2	owns the fd, acquires, publishes, sends the sample event
`TELE`	3	reads the surface, drives the sweep, writes USART2
`CTRL`	4	blocks in `read()` on the console, applies commands

`TELE` sits *below* `SAMP` on purpose. A higher-priority reader can preempt a publish in progress and burn its retry budget for nothing; a lower-priority one is guaranteed a stable payload, and still gets the CPU because the sampler sleeps inside every conversion wait. Symmetrically, the sampler yields a tick after a *failed* cycle — without that, a disconnected sensor turns it into a busy-loop at priority 2 and nothing below it runs to report the fault.

`CTRL` sits below both: it is blocked in `read()` almost always, and an arriving command must not preempt a publish that `TELE` is reading. A console returning EOF or an error would spin the loop, so that case sleeps 100 ms.

The three validation captures of Section 3.1 predate the priority being set to 4: they were taken with `CTRL` created at 2. No command was typed during any of them, so the task stayed blocked in `read()` and never became runnable, and the measured timings are those of the two tasks above it.

Hardware setup. The examiner needs three connections: the sensor on I2C1, the USB-serial adapter on USART2, and the ST-Link (the board's own on-board USB) for flashing.

Signal	STM32F4-Discovery pin	Peer
SCL	PB6	BMP180 SCL
SDA	PB7	BMP180 SDA
VDD (3.3 V)	3V	BMP180 VIN
GND	GND	BMP180 GND
Console TX	PA2	CP2102 RXD
Console RX	PA3	CP2102 TXD
GND	GND	CP2102 GND

I2C1 runs at 100 kHz standard mode, sensor address `0x77`. The console is USART2, 115200 8N1.

Toolchain and BSP configuration. Full step-by-step instructions are in `B_docs/SETUP.md`; the short version is: build the `7/rtems-arm` toolchain with the RTEMS Source Builder, then configure and install the `arm/stm32f4` BSP with `waf`.

One BSP option is not optional for this project. The stock `arm/stm32f4` BSP puts the console on USART3; we use USART2, so the BSP must be built with the following `config.ini` before `./waf configure`:

```
[arm/stm32f4]
BUILD_TESTS = True
STM32F4_ENABLE_USART_2 = True
STM32F4_ENABLE_USART_3 = False
```

The resulting setting can be confirmed in the installed `.../arm-rtems7/stm32f4/lib/include/bspopts.h`, which should show `STM32F4_ENABLE_USART_2` defined and `STM32F4_USART_BAUD 115200`.

Building and flashing. The toolchain path is written in exactly one place, `.env/setup.env`, a single `KEY=VALUE` line copied from the committed example. `compile.sh`, `flash.sh`, `Makefile` and the root `CMakeLists.txt` all read that file, so the four build paths cannot drift.

```
cp .env/setup.env.example .env/setup.env
# edit it: RTEMS_LOCAL_PATH=/path/to/where/RTEMS_toolchain/lives

cd C_src
./compile.sh        # or: make        -- builds the .exe
make flash          # build + program the board over OpenOCD
```

Everything is compiled with `-std=c++17 -Wall -Wextra -Werror -fno-exceptions -fno-rtti` for Cortex-M4 with hard-float (`-mfpu=fpv4-sp-d16`). Flashing goes through OpenOCD with `board/stm32f4discovery.cfg`, programming the raw binary at `0x08000000`.

Running it. Attach a serial terminal to the USB-serial adapter and reset the board (black B2 button, or let `capture.sh` reset it over the ST-Link):

```
screen /dev/cu.usbserial-0001 115200     # macOS: cu.*, never tty.*
                                         # exit screen: Ctrl-A then \
```

A healthy boot looks like this:

```
[SELFTEST] compensate: T=150 (exp 150)  P=69964 (exp 69964)  -> PASS
#BMP180 v1 fw=2.0.0 temp_ms=1000
[[DEBUG]] BMP180 registered on /dev/bmp180-0
S 1043221 244 96822 0
S 1054220 244 96825 0
```

The first line is a hardware-independent self-test of the Bosch compensation math, run at every boot against the datasheet worked example; a `FAIL` there means the arithmetic is broken, independently of any wiring problem. The firmware then registers the bus and the sensor and starts streaming.

Using the driver from your own task. Two ways, depending on whether you want to own the device:

```
/* Through the application layer -- the normal path. Non-blocking,
   no bus traffic, no descriptor. */
bmp_app_sample_t s;
if (bmp_app_read(&s)) { /* s.pressure_pa, s.temperature_cdeg, s.oss, s.seq */ }
bmp_app_set_oss(BMP180_OSS_ULTRA_HIGH_RES);   /* applied next cycle */
bmp_app_request_soft_reset();                 /* power-on defaults */

/* Directly on the device node -- only if nothing else owns it. */
int fd = open("/dev/bmp180-0", O_RDWR);
bmp180_measurement_t m;
ioctl(fd, BMP180_IOCTL_READ_MEASUREMENT, &m);
```

3 Project outcomes

3.1 Concrete outcomes

The firmware. Repository https://github.com/AlbertoFurlan20/ES2025, nine firmware releases from 1.0.0 to **2.0.0**.

Release	What changed	Interval (ms)
1.0.0	first working driver: polled I2C master, sensor driver, ioctl ABI	11/14/20/32
1.1.0	on-device statistics replaced by the telemetry stream; measurement path byte-identical	11/14/20/32
1.2.0	unreachable code deleted; eight bugs closed, seven by deletion	11/14/20/32
1.2.1	one tick of padding on every conversion wait	13/16/22/34
1.3.0	`oss` validated at registration, compensation divide guarded	13/16/22/34
1.4.0	SCO polling replaces the worst-case sleep; 1 s temperature cache; timed I2C waits	5/7/11/19
1.5.0	I2C bus recovery for a slave holding SDA low	5/7/11/19
1.6.0	application layer added, so more than one task can use the sensor	not captured
2.0.0	the application layer becomes the only path to the device; emitter task and ring removed; console control task added	5/7/11/19

The last column is the median acquisition interval for `oss` 0 to 3, recomputed from the committed captures. It moves in three steps and every other release holds it, which is the check each of those releases had to pass. 1.6.0 is the one release with no capture of its own: it was superseded by 2.0.0 before a sweep was recorded, and it is also the one version never tagged.

Host-side tooling and data. `E_analysis/` is a small Python package split so that the metrics are testable without hardware or a display: `parse.py` (stream to DataFrames), `metrics.py` (DataFrames to numbers), `plot.py` (numbers to charts), plus `capture.sh`, which resets the target over the ST-Link and writes a timestamped log, plus `make_report_figures.py`, which renders the figure set used in this report. The repository carries 44 captures taken during development, including the ones that record failures, so every sweep, timing and recovery number quoted here can be recomputed from committed data without a board.

Tests. Two host-compiled suites that need no toolchain and no board (`make -C C_src/tests run`): the telemetry formatter, and the seqlock snapshot including a concurrent writer. On the target, the compensation self-test runs at every boot, and three build flags select fault and teardown tests that must not be present in a normal build:

```
-DBMP180_TEARDOWN_TEST    -DI2C_RECOVERY_TEST    -DBMP180_NO_BUS_LOCK
```

Validation. `C_src/TESTING.md` is a ten-step validation ladder, from hardware-independent math up to physical and fault tests, and each step states what to expect and what a failure means. The quantitative core is an oversampling sweep: 500 samples in each of the four modes after three discarded warm-up samples, then a continuous run at `OSS=2`. No statistics are computed on the device.

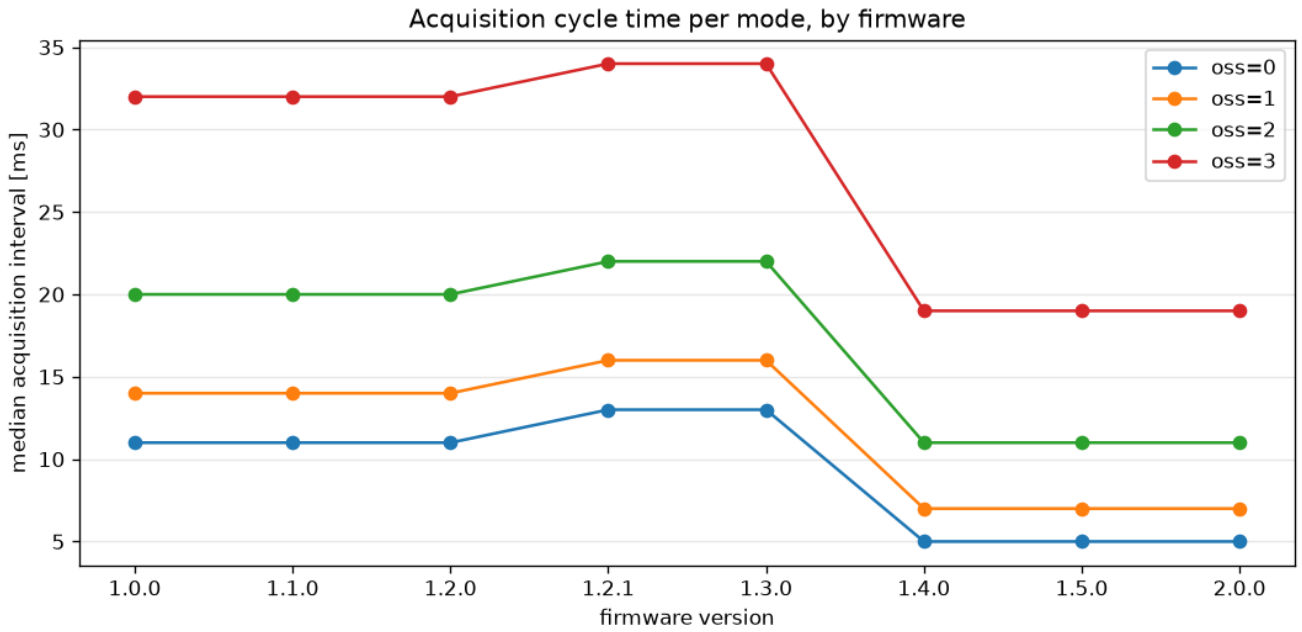
Release 2.0.0 was validated on hardware with **three consecutive 150 s captures**, committed as `E_analysis/captures/2026090-170059.log` and `-170351.log`. Each carries about 13 640 samples, and all three report **zero errors, zero dropped records and zero malformed lines**. Zero drops is what shows that emission never throttled acquisition, the premise every timing figure rests on.

The acceptance gate for the release was that the four median acquisition intervals be unchanged from the previous one, since the restructuring touched who may talk to the device and not the acquisition path. They were identical in all three runs and identical to the figures recorded for 1.5.0:

`oss`	n per sweep block	median interval (μs)	mean pressure, run 1 (Pa)	datasheet RMS (Pa)
0	500	5000	100545.9	6.0
1	500	7000	100545.9	5.0
2	500	10999	100543.5	4.0
3	500	18999	100542.3	3.0

The interval column is the same in all three runs; the mean-pressure column is run 1 alone, since absolute pressure moves between runs with the weather. Within a run, mean pressure is constant across the four modes to within 4 Pa in run 1 and 7 Pa in the worst of the three, as it must be: the true pressure does not depend on how the sensor is sampled.

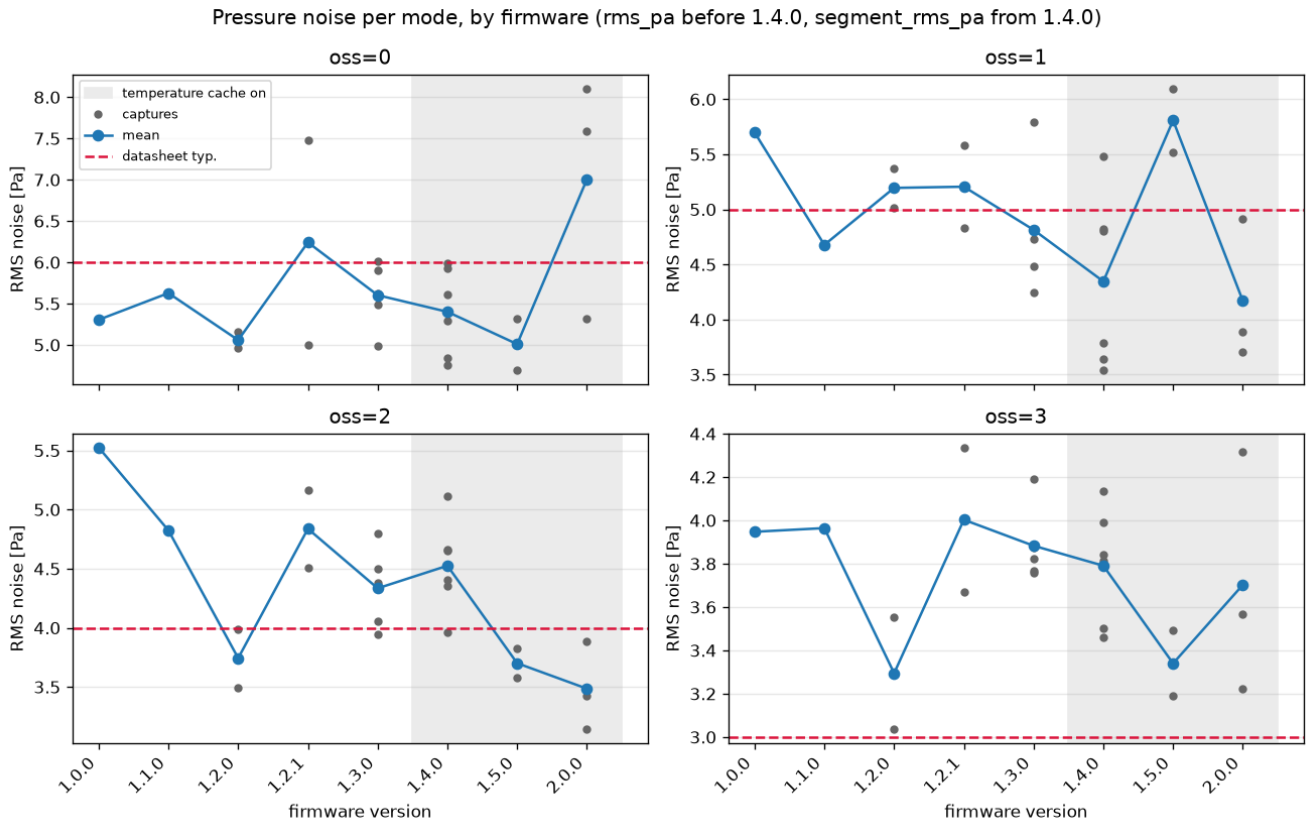
The measured history. Every release was captured the same way, so the whole development history can be read off the committed data. Twenty-two captures recorded a complete sweep across eight firmware versions; the remaining committed captures are deliberate failures (wedged buses, resets landing mid-transfer) and are excluded, since a partial sweep is not comparable. One of the two 1.5.0 points is the forced bus-recovery build of Section 3.4 rather than the release binary; it is kept because its four intervals are identical to the release's, the check that build exists to pass.



Acquisition cycle time has three regimes, and each step is a decision recorded in `CHANGELOG.md`:

- **1.0.0 to 1.2.0 — 11 / 14 / 20 / 32 ms.** The driver slept the datasheet maximum for the mode and then read the result registers.
- **1.2.1 to 1.3.0 — 13 / 16 / 22 / 34 ms.** Two milliseconds *slower* per sample, deliberately. Every conversion constant gained one tick of padding, because `rtems_task_wake_after(n)` blocks for between $n-1$ and n ticks and the unpadded waits were occasionally expiring while the sensor was still converting: throughput paid for correctness.
- **1.4.0 onwards — 5 / 7 / 11 / 19 ms.** Between 1.7 and 2.6 times faster depending on the mode, from two changes at once whose contributions separate cleanly. The temperature cache removes one whole conversion, which is the same 6 ms whatever the mode, so it dominates the *ratio* at `oss`=0 where the cycle was shortest to begin with. SCO polling saves the gap between a conversion's padded maximum and its typical time, which grows with the mode (3 ms at `oss`=0 against 10 ms at `oss`=3), so it dominates the *absolute* saving at the top mode, where 15 of the 34 ms disappear. Polling the SCO bit replaced the fixed worst-case sleep, so a cycle ends when the conversion ends rather than at its datasheet maximum, and most cycles skip the temperature conversion entirely. The padded maxima became the timeout rather than the wait.

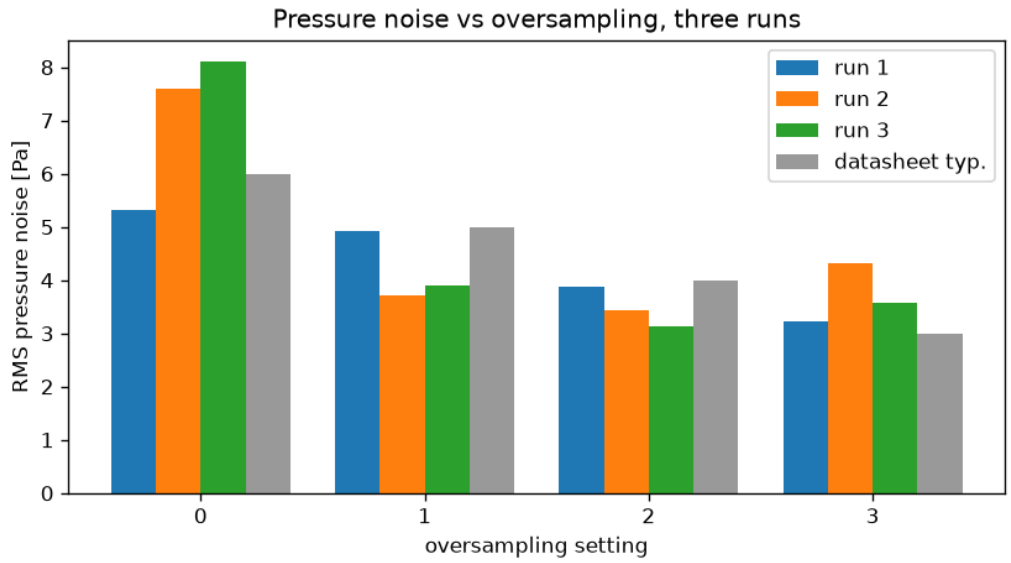
Release 2.0.0 lands on the 1.4.0 line: moving every consumer off the device and behind an application layer cost the acquisition path nothing.



Noise shows no such steps. Across all eight versions each mode's per-version mean stays inside a band at most about 2 Pa wide, near its datasheet figure, with no trend: the firmware changes moved timing, structure and fault behaviour, none of which make the sensor quieter or noisier. The two eras are measured with different columns, whole-block RMS before the temperature cache existed and median within-segment RMS after it, and the chart marks the boundary rather than blending them: with the cache off the segments are two or three samples long and their spread understates the noise. The one visible outlier, `oss=0` at 2.0.0, is the post-boot settling transient identified below.

Pressure noise is reported per mode as the median within-segment RMS, not the whole-block RMS. The driver compensates every reading in an interval against one cached temperature, and compensation moves about 25 Pa per $0.1\,^{\circ}\mathrm{C}$, so a drifting temperature leaves the stream as a staircase and whole-block RMS would measure thermal drift times the cache interval rather than the sensor. Across the three runs:

oss	run 1 (Pa)	run 2 (Pa)	run 3 (Pa)	datasheet typ. (Pa)
0	5.32	7.59	8.11	6.0
1	4.91	3.71	3.89	5.0
2	3.89	3.43	3.14	4.0
3	3.22	4.32	3.57	3.0

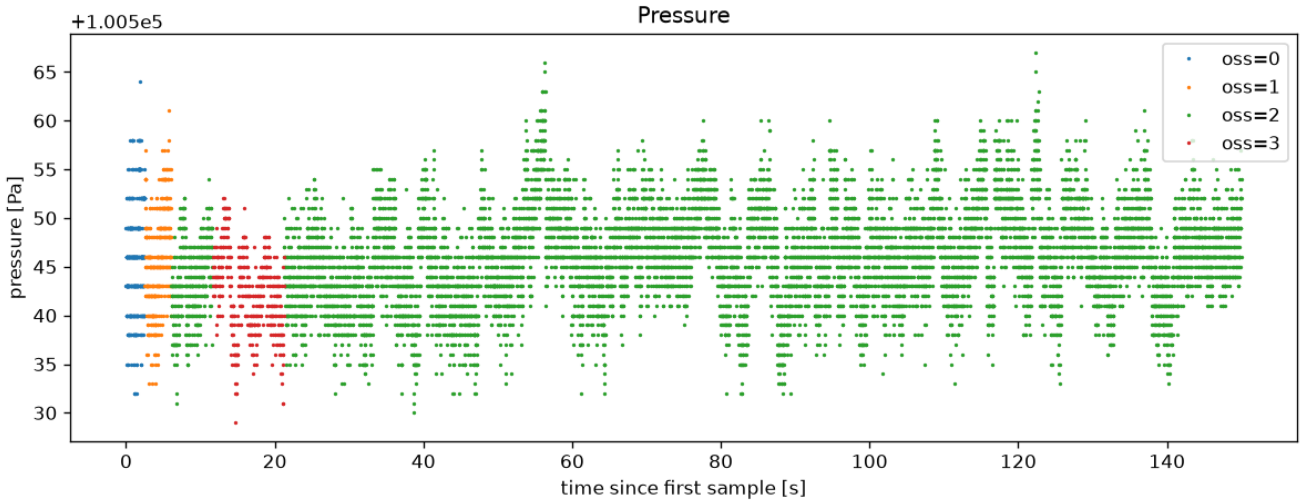


Oversampling works: noise falls by roughly a factor of two from oss=0 to oss=2 in every run. We do not claim the monotonicity criterion as passed, though. It holds across the first three modes in all three runs; at the oss=2 to oss=3 step it holds in one run of three.

The obvious explanation does not survive. Since the sweep visits each mode once in sequence, a block's exposure to atmospheric movement grows with the mode (2.5 s of wall clock at oss=0 against 9.5 s at oss=3), so we fitted and removed a linear pressure trend from each block. At oss=3 detrending changes almost nothing (3.22, 4.32 and 3.57 Pa become 3.92, 4.14 and 3.99 Pa), so within-block drift is *not* what puts that mode above oss=2. What remains is a difference of well under 1 Pa against a datasheet separation of 1 Pa and run-to-run scatter of the same order: the experiment as designed cannot resolve the top step, and interleaving the modes within a sweep, so that every mode is compared against the same drift, is the change we would make before trusting it.

The same test does explain the other anomaly. oss=0 reads above its datasheet figure in two runs of three, and it is the first block after boot: its fitted trend is -6.0 and -7.7 Pa/s in those two runs, an order of magnitude steeper than any other block. Detrended, those blocks read 6.19 and 5.90 Pa against a datasheet 6.0: a settling transient the sweep rides through, not sensor noise, and an argument for discarding more than three warm-up samples.

The profile is visible directly in the stream: the three short sweep blocks, the long continuous run at oss=2, and the compensation staircase produced by the one-second temperature cache.



Fault behaviour was verified on earlier builds of the same sampler and is quoted as such. Pulling SDA during a run, measured on the build immediately preceding the current structure and recorded in `TESTING.md` section 10 (that capture file is not among the committed ones), produced one `E 5 (EIO)` record 9 995 μs after the last sample; samples stopped rather than repeating with fresh timestamps, and the stream resumed on reconnect with no reset, short by the outage divided by the cycle time (13 088 samples against a 13 663 baseline, zero drops). Bus recovery was verified both ways on 1.5.0: forced on a healthy bus (`20260830-131247`) it leaves the timing unchanged, and on a genuinely wedged bus (`20260830-131424`, `-131457`) it recovers on the next boot. The same failure on 1.4.0 killed three consecutive boots (`20260830-131609` to `-131611`, zero samples) and required reflashing to clear.

3.2 Learning outcomes

Because we arrived knowing the buses and the device, almost everything we learned sits on the operating-system side of that line. What changed is not that we can now read a datasheet or drive an I2C transfer, but that we treat both as contracts a scheduler has to be designed around.

- **How a real-time kernel's device model fits together.** What it means for an IMFS node to *own* a device, why the teardown order carries the safety argument, how an ioctl ABI is designed so it can be frozen, and how a bus driver and a device driver meet through a framework interface rather than through each other. The Linux driver model gave us the vocabulary; RTEMS taught us how little of it transfers unchanged.
- **That knowing a protocol is not the same as owning a master.** We could read the I2C waveform before we started and still had to write 622 lines of clock gating, alternate-function mux and CCR clock math to produce one. The lesson that did not come from the protocol is the wedge: a slave holding SDA low survives every reset, no amount of retrying clears it, and recovery has to be designed in rather than discovered.
- **That lock-free is a scheduling decision before it is a performance one.** The published snapshot is a seqlock because an RTEMS mutex held by a low-priority reader would block the acquisition task until the priority-inheritance handoff completed. The same reasoning sets the task priorities, and getting it wrong cost a measured 2.70 s console blackout.

Tools we learned to use. The RTEMS Source Builder and `waf` for building a cross toolchain and a BSP; the `arm-rtems7` compiler, `objcopy` and `objdump`; OpenOCD and the ST-Link for flashing and for resetting the target from a script; `screen` and raw termios programming for the serial console; `pandas` and `matplotlib` for the host-side analysis; and Git branching and semantic versioning applied to firmware, where a version number has to mean something about the measurement path.

3.3 Existing knowledge

Both of us started with a solid embedded background: C, the Linux kernel and its driver model, shell scripting for build and capture automation, and the serial buses this project rests on — UART/USART, I2C and SPI — including reading timing diagrams and datasheets as specifications rather than reference material. Sensor interfacing at the register level was familiar territory, so the BMP180 calibration arithmetic and its conversion-time table were a known kind of problem. Robotics work, and ROS in particular, had also made sensor data familiar as something published, timestamped and consumed by someone else, which is where the telemetry record format and the snapshot-publishing model came from.

That background covered the device and the bus, not the operating system around them. RTEMS itself, the RSB and `waf` build path, the BSP configuration surface, and the classic API's tasks, semaphores and rate-monotonic periods were new to us, as was the discipline of a system with no debugger and no printf that does not perturb what it measures. ROS had given us publishers and subscribers but hidden every question this project is about: there was no middleware here to own the buffer, the timing or the priorities. The gap between what we knew and what the project needed sat almost entirely on the RTOS side.

3.4 Problems encountered

- **The BSP ships no I2C bus driver for the framework the device driver targets.** We found this only after the sensor driver was written: there was nothing for `i2c_dev_alloc_and_init()` to attach to. It roughly doubled the scope of the project; the 622-line polled STM32F4 master in `i2c/` exists because of it.
- **Every `open()` returned `ENFILE`.** The RTEMS default for `CONFIGURE_MAXIMUM_FILE_DESCRIPTORS` is 3, and `stdin`, `stdout` and `stderr` consume all three, so opening the bus node and the device node was impossible. The symptom ("too many open files" on a system with two files) looked nothing like the cause.
- **The console was silent for hours.** Two independent causes stacked: the stock BSP puts the console on USART3 while we had wired USART2, and on macOS the port must be opened as `/dev/cu.*` and never `/dev/tty.*`. A related trap cost another afternoon: termios settings applied with `stty -f` to a `cu.*` node are reset when the port is opened, so `cat` read at the driver's default baud and recorded framing garbage. The capture tool therefore applies them to an already-open descriptor and holds it for the whole run.
- **A BSP header uses `or` as an identifier.** It compiles as C and cannot compile as C++, where `or` is an alternative spelling of `||`. Not a problem any of us had met before.
- **Conversion waits expiring one tick early.** `rtems_task_wake_after(n)` blocks for between $n-1$ and n ticks, because the call lands somewhere inside the current tick. Our conversion timeouts used the datasheet maxima unpadded, so the driver occasionally read the result registers while the sensor was still converting, and still within specification. The defect was stochastic and invisible in any single reading; what exposed it was that RMS noise did not fall monotonically with oversampling, which it must. The fix is one tick of padding on every conversion constant, and the header says not to "correct" them back.
- **The board came up dead after a reset, and stayed dead.** A CPU reset landing mid-transfer leaves the sensor holding SDA low; the peripheral latches BUSY and every transfer fails, across every subsequent reset. Only a power cycle cleared it. It reproduces in roughly two boots out of seven once bus occupancy is high, and the fix is the recovery sequence of Section 2.1.
- **A failing sampler starved the task that was supposed to report the failure.** On a successful cycle the sampler sleeps inside the conversion wait and yields; on a failing one the ioctl returns immediately, so a disconnected sensor turned it into a busy-loop at priority 2. Pulling SDA during a capture produced 2.70 s of complete console silence, no samples, no errors and no drops, because nothing below it could run. The sampler now yields a tick after a failed cycle, which is also the slot the consumer uses to notice and report the errno.
- **A stale reading that looked fresh.** An early version stamped the last good measurement with the current timestamp when a cycle failed, so a dead sensor produced a plausible, perfectly-paced stream of a constant value, with nothing in the data to say anything was wrong. The layer now refuses to republish on a failed cycle, so the

timestamp freezes and the age of the data stays visible. The header documents it, since the check cannot be forced on the consumer.

- **Aliasing between the temperature cache and the noise statistics.** Compensation moves about 25 Pa per $0.1\,^{\circ}$C, so compensating a second of pressure readings against one cached temperature leaves the stream as a staircase: flat inside an interval, stepping when the cache refreshes. Whole-block RMS then measures thermal drift times the cache interval rather than sensor noise: 16.21 Pa against 4.85 Pa within-segment in one measured block. The analysis now splits each block at the cache refreshes and reports the median within-segment RMS.
- **A use-after-free in teardown.** Freeing the device directly leaves a published IMFS node pointing at freed memory. The correct order is `unlink()`, which runs the node's destroy handler. Nothing in a normal run ever tears the device down, so the whole path was dead code, which is where the bug lived; it is now exercised under a build flag that registers, opens, unlinks and re-opens once at boot.

4 Honor Pledge

(**This part cannot be modified and it is mandatory to sign it**)

I/We pledge that this work was fully and wholly completed within the criteria established for academic integrity by Politecnico di Milano (Code of Ethics and Conduct) and represents my/our original production, unless otherwise cited.

I/We also understand that this project, if successfully graded, will fulfill part B requirement of the Advanced Operating System course and that it will be considered valid up until the AOS exam of Sept. 2022.

Group Students' signatures